

ProfileSvr Handbook

The product requirements and the complete API reference for the profile microservice: OTP-verified onboarding, device-bound identity, KYC with face liveness to tier 1, automatic banking-account provisioning (NGN · CAD · virtual), virtual-account credit settlement, transaction PIN, and credential recovery — everything a client team or reviewer needs in one page.

Base URL http://localhost:5107	Envelope {isSuccess, data, message}	
Auth Bearer JWT (SSO-issued)	Stack .NET 9 · MySQL · vertical slices	Tests 67 passing
1 · Product overview what this service is for	2 · User journeys four paths through the product	3 · Functional requirements FR-1 ... FR-24
4 · Security model token types · OTP discipline · device trust	5 · Data & integrations tables · SSO · Message Centre · KYC	6 · Banking accounts provisioning · credit settlement · providers
7 · Risks & metrics open items · success measures	8 · API conventions envelope · auth · encryption	9 · Onboarding API 8 endpoints incl. KYC liveness
10 · OTP verification the single verify endpoint	11 · Auth API login · tokens · password · device change	12 · Profile API reads · PIN · audit feed
13 · Webhooks virtual-account credits in	14 · Appendix OTP purposes · KYC statuses · status codes	

What ProfileSvr is

ProfileSvr is the identity and onboarding backbone for the Muuvly app. It owns the customer profile, the devices a customer signs in from, and every one-time-passcode ceremony that proves who is acting — while delegating credentials to the central SSO and message delivery to the Message Centre.

Goals. A new customer reaches a fully verified, transaction-ready profile in one guided flow; every sensitive change (device, PIN, password) demands proof beyond a password; every action leaves an audit trail with device and IP; and a downstream transaction service can read risk signals (tier, fresh-device gate, PIN state) from a single DTO.

Non-goals. ProfileSvr does not store passwords or mint tokens (the SSO does), does not deliver messages (the Message Centre does), and does not execute transactions — it only supplies the identity facts transactions are gated on.

One sentence: the SSO knows *who you are*, ProfileSvr knows *everything else about you* — and nothing about you changes without an OTP.

Four journeys

JOURNEY	TRIGGER	PATH	END STATE
Onboard (manual)	New customer installs the app	initiate → verify-auth → initiate-phone → otp/verify · Phone → create-profile	Active , tier 0, device bound
Onboard (KYC)	Customer has a BVN/NIN	initiate → verify-auth → initiate-kyc (identity lookup + liveness session) → otp/verify · Phone → complete-kyc (selfie ↔ ID photo match) → create-profile	Active , tier 1 , identity + phone verified from the BVN/NIN record, NGN + CAD + virtual accounts provisioned
Return & act	Daily use	login (or /me on relaunch) → PIN & password self-service under a profile-active token	Session with full profile DTO, live account balances, exchange rates, and risk gates
Recover	Lost/new phone, or interrupted signup	Device switch: initiate-device-change → change-device. Resume: login → initiate-resume → resume	New device bound (24 h reduced-limit gate) or record rebuilt under the original id

Requirements

Onboarding

- FR-1 Onboarding starts with an email and a device; the service pre-allocates the identity (`sourceId`) that becomes both the SSO user's `SourceId` and the profile id.
- FR-2 The email is proven by a 6-digit OTP before any account exists; `verify-auth` then creates the SSO auth and a staging record (`status: AuthCreated`) and returns onboarding-typed tokens so no separate login is needed.
- FR-3 The phone number is proven by an OTP delivered over WhatsApp (SMS switchable by config); in the KYC path the phone comes from the BVN/NIN record, never from the client.
- FR-4 The profile is only *created* (status `Active`) at the final step, after both email and phone are verified and personal details are supplied.
- FR-5 KYC is proven biometrically: an AWS face-liveness session opens at `initiate-kyc` , and `complete-kyc` compares the liveness selfie with the BVN/NIN photo — a match marks the id verified, sets `kycStatus: Approved` , promotes to tier 1 and provisions the banking accounts. A failed liveness or mismatch is a soft failure the client retries with a new session.
- FR-6 Re-initiating before the profile exists simply resends a fresh OTP (60 s cooldown), invalidating earlier codes.

Devices

- FR-7 A profile may accumulate many devices over time, but exactly **one is in use** at any moment — and an in-use device serves exactly one profile. Released bindings are kept as history.
- FR-8 Devices are registered *only* inside OTP-protected flows (onboarding, resume, device change); there is no standalone registration endpoint.
- FR-9 Login refuses any device other than the active one (403); only the OTP ceremonies move a binding.
- FR-10 A locked-out user (new phone, no token, no profile id) switches devices with password + email OTP alone: `initiate-device-change` → `change-device` .
- FR-11 The active binding's `linked_at` opens a **24-hour reduced-limit gate** (`deviceRecentlyLinked`) that a transaction service reads live; it expires by arithmetic, not by a job.
- FR-12 An SSO account with no local record (interrupted signup) resumes via its own OTP ceremony, reclaiming the original profile id from the token's `SourceId` claim; a device in use by another profile cannot be claimed through resume.

Credentials & PIN

- FR-13 Login is an SSO password grant gated on the registered active device; it returns tokens, a `deviceStatus` (New / Existing / Unlinked), the token `type` , and the full profile DTO.
- FR-14 Sessions refresh without a password (`/auth/refresh`); app relaunch rehydrates from the token alone (`/auth/me`).

- FR-15** Password change requires the current password; password reset requires an email OTP; both execute on the SSO.
-
- FR-16** The 4-digit transaction PIN is set once, changed with the current PIN, or reset via email OTP; it is stored as PBKDF2-SHA256 with a per-profile salt and is verified during transactions, not by this API.

Banking accounts & money-in

- FR-25** Once the identity (BVN/NIN) is verified, the service provisions the banking chain automatically: customer (CIF) → NGN account → CAD account → virtual (collection) account — each step get-or-create and best-effort, never blocking the caller.
-
- FR-26** Failed provisioning surfaces as `kycStatus: ProvisioningFailed` and **self-heals**: create-profile, login and refresh each retry the missing pieces until the chain completes.
-
- FR-27** The bank behind the accounts is replaceable: endpoints only see a facade, with the core-banking provider (OneCore today) and the virtual-account provider (VantPay today) swappable behind two interfaces.
-
- FR-28** Login and refresh return the live account list (with balances) and the latest exchange rate per currency pair; provider outages yield empty lists, never a failed login.
-
- FR-29** Incoming virtual-account credits arrive on a signed webhook, are stored idempotently by `transactionRef`, and a background job settles them into the CBA naira account — with configured platform and provider charges — retrying transient failures and dead-lettering permanent ones.
-
- FR-30** The credit-posting job carries a heartbeat surfaced on `/health`; a stalled job degrades health. A MySQL advisory lock keeps multiple replicas from double-posting.

Platform

- FR-17** Every response uses the `{isSuccess, data, message}` envelope with a human-readable message, always populated.
-
- FR-18** Every significant action writes an activity row: type, profile, device, client IP, timestamp — readable by the owner as a paged feed.
-
- FR-19** Tokens carry a `type` claim (onboarding · temporary · profile-active) written to the SSO; ProfileSvr re-derives it per request from the live profile status so a stale token can never cross a gate.
-
- FR-20** Onboarding-typed tokens reach only onboarding endpoints; profile-active tokens reach everything except onboarding; crossing the line is 403.
-
- FR-21** OTP emails are branded HTML (configurable brand name); WhatsApp/SMS are plain text; all sends verify the Message Centre's response body, not just the HTTP status.
-
- FR-22** Optional end-to-end payload encryption (AES-256-GCM envelope) can be switched on by config; docs and health stay plaintext.
-
- FR-23** BVN and NIN are stored raw but always masked to the last 4 digits in every API response.
-
- FR-24** Schema changes apply automatically at startup via EF Core migrations.

Three layers of proof

Token types

TYPE	WHEN	MAY REACH	MAY NOT REACH
temporary	Token authenticates, no profile row	resume bootstrap, /me, otp/verify	everything else
onboarding	Profile AuthCreated	initiate-phone, initiate-kyc, create-profile (+ shared)	PIN & password endpoints
profile-active	Profile Active	everything except onboarding	onboarding endpoints

The claim lives in the JWT (written to the SSO before token mint) for other services to read — but ProfileSvr always recomputes it from the database per request, so the gate flips the instant `create-profile` succeeds, with no stale-token window.

OTP discipline — identical everywhere

- 6-digit cryptographically random code; public 16-character `retrievalCode` reference (no 0/O/1/I).
- Stored as `SHA-256(code | purpose | target)` — a code can never verify outside the exact ceremony and recipient it was minted for.
- 10-minute expiry · 60-second resend cooldown · **5 attempts then locked** · one active code per purpose · superseded codes invalidated.
- If the send fails after issuing, the orphaned code is consumed and the caller gets a 502 — no live code without a delivered message.

Everything else

- **PIN:** PBKDF2-SHA256, 100k iterations, fresh random salt per set; constant-time compare.
- **Device trust:** password proves the account, the mailbox OTP approves the device — stolen credentials alone cannot enroll an attacker's device.
- **Payload encryption** (optional): AES-256-GCM, fresh nonce per message, authenticated — a flipped byte is rejected, never garbled.
- **Fail closed:** SSO or KYC unreachable → 503 before anything persists.
- **Webhook integrity:** inbound credit notifications must carry `X-Signature` = HMAC-SHA256 of the raw body (fixed-time compared); outbound CBA postings are signed the same way.
- **Biometric proof:** tier 1 requires a live selfie matched against the BVN/NIN photo — a stolen id number alone cannot pass KYC.

Data model & dependencies

TABLE	HOLDS	KEY COLUMNS
profiles	The customer record and every status flag	status, email/phone + confirmations, names, DOB, gender, tier, address, masked BVN/NIN + verifications, kycStatus (+ reason), cif, naira/CAD/virtual account numbers, PIN salt/hash, device-change gate timestamps
devices	Registered devices with hardware metadata	unique deviceId, name, platform, OS, manufacturer, model, appVersion
user_devices	Profile↔device bindings; one active per side, history kept	linked_at_utc (the 24 h gate clock), released_at_utc
otp_codes	Every OTP ever issued	retrieval_code (unique), purpose, target, bound hash, attempts, expiry, consumption, pre-allocated source_id
activities	Immutable audit trail	type, profile, device, ip_address, timestamp
virtual_account_mappings	Virtual (collection) account → CBA naira account routing	virtual_account (unique), cba_account, profile, account name/bank
virtual_account_credits	Every inbound credit notification, and its settlement state	transaction_ref (unique), virtual_account, amount, sender details, credit_posted, attempts, posting_abandoned, provider response

DEPENDENCY	USED FOR	CONTRACT NOTES
SSO (OpenIddict)	Accounts, password/refresh grants, JWKS validation, custom claims	Pre-registration calls authenticate with a client-credentials <i>service client</i> ; claim create replaces in place; SourceId claim = profile id
Message Centre	Email (HTML template), WhatsApp, SMS	Queue-based; response body's <code>isSuccessful</code> is checked; senders (<code>EmailFrom</code> , <code>WhatsApp from: 91096</code>) are required for actual delivery; phone numbers sent digits-only
KYC provider (DigitKYC)	BVN/NIN lookup → identity, phone, photo	Pluggable HTTP client behind <code>IKycClient</code> ; deterministic mock in dev (numbers ending 00 = not found)
AWS Rekognition	Face-liveness sessions and selfie ↔ ID-photo comparison	Behind <code>IFaceVerificationService</code> ; mock switchable via <code>AWS:UseMockFaceVerification</code>
OneCore (core banking)	Customer (CIF), NGN + CAD deposit accounts, exchange rates, credit posting	Behind <code>IAccountProvider</code> ; client-credentials token against its own SSO; product codes per currency in config

DEPENDENCY	USED FOR	CONTRACT NOTES
VantPay (DigitVirtual)	Static virtual (collection) accounts + inbound credit webhooks	Behind <code>IVirtualAccountProvider</code> ; webhook signed with HMAC-SHA256 when <code>DigitVirtual:WebhookSecret</code> is set

6 · BANKING ACCOUNTS

Accounts & money-in

Once KYC approves an identity, ProfileSvr turns the profile into a banked customer without any further client calls — and once the customer shares their virtual account number, incoming transfers settle into their naira account automatically.

The provisioning chain

Runs first at `complete-kyc` (the provider requires a verified BVN/NIN), and re-runs at `create-profile`, `login` and `refresh` until complete:

- **1 · Customer (CIF)** — searched by email on OneCore first, created only if absent (idempotent).
- **2 · NGN account**, then **3 · CAD account** — existing accounts in the right currency are adopted; otherwise opened with the configured product codes.
- **4 · Virtual account** — a static VantPay collection account tied to the BVN/NIN, opened only after the NGN account exists so every credit has somewhere to land; its routing row (`virtual` → `naira`) is written in the same transaction.

Every step is best-effort: a provider outage logs a warning, leaves `kycStatus: ProvisioningFailed` ("we'll retry automatically"), and the next login or refresh picks up where it stopped. The caller's request never fails because a bank was down.

The money-in pipeline

- **Notify** — the provider calls `POST /webhook/virtual-account-credit` ; the signature is verified and the credit stored, idempotent on `transactionRef` (redeliveries and races are absorbed).
- **Settle** — the `PostCbaCreditsJob` sweeps stored `SUCCESSFUL` credits every ~2 s (batch 1000), resolves the `virtual` → `naira` mapping, computes the platform charge (`ChargePercent`) and provider charge (`ProviderChargePercent`), and posts a signed credit to OneCore's integrations webhook.
- **Converge** — a CBA "duplicate reference" reply counts as settled; transient failures retry each tick up to `MaxPostAttempts` ; permanent (4xx) failures dead-letter immediately with the error preserved.
- **Observe** — the job beats a heartbeat that `/health` reports; a stalled job flips health to `degraded` . Replicas coordinate through a MySQL advisory lock.

Configuration surface

SECTION	DRIVES	KEYS
OneCore	Core banking + credit posting	BaseUrl, SsoBaseUrl, ClientId/Secret, OfficeId, AccountOfficerId, NairaProductId, CadProductId, WebhookUrl, TransferIntegrationKey/Secret, VirtualAccountCreditNarration
DigitVirtual	Virtual accounts + webhook auth	BaseUrl, SsoBaseUrl, ClientId/Secret, BankName, WebhookSecret
VirtualAccountCredit	The settlement job	Enabled, BatchSize, IntervalSeconds, MaxPostAttempts, ChargePercent, ProviderChargePercent
AWS	Face liveness	AccessKey, SecretKey, Region, UseMockFaceVerification

When OneCore / DigitVirtual base URLs are unset, deterministic mocks provision fake accounts so local development runs end-to-end — never ship the mocks to production.

7 · RISKS & METRICS

Open items, risks, and what success looks like

RISK / OPEN ITEM	IMPACT	MITIGATION / OWNER
WhatsApp delivery depends on the Message Centre's queue consumer and an approved business template	Phone leg blocks	SMS channel switch in config; MCS team owns template approval
OneCore / VantPay / AWS credentials unset fall back to mocks or empty results	Fake accounts in dev could be mistaken for real; empty balances after login	Fail startup in production when a required provider is unconfigured
An unset <code>DigitVirtual:WebhookSecret</code> skips signature verification	Forged credit notifications would be stored (though not settled without a mapping)	Make the secret mandatory outside dev
Charge percentages live in config with no audit of applied values per credit	Reconciliation disputes hard to settle	Persist the computed charge on each credit row
Message Centre reports "queued", never "delivered"	Silent drops invisible	Request delivery callbacks/status from MCS team
Payload-encryption key is a shared symmetric secret	Rotation requires an app release	Evolve to per-device ECDH negotiated at onboarding
Secrets load from a gitignored <code>appsettings.Secrets.json</code> or env vars	Leak risk if the pattern is bypassed	Move to a managed secret store before production; rotate anything ever committed

PIN endpoint has no attempt lockout (verification happens in the transaction service)

Depends on downstream throttling

Mirror the OTP 5-attempt rule where the PIN is verified

Success metrics

- **Onboarding completion:** initiate → Active conversion, and time-to-Active.
- **OTP health:** delivery success per channel, verify-on-first-attempt rate, lockout rate.
- **Recovery:** device-switch and resume completion without support contact.
- **Risk:** share of transactions inside the 24 h fresh-device gate; KYC tier-1 uptake; liveness pass rate and face-match rate.
- **Money-in:** credit settlement latency (webhook → CBA posting), retry depth, dead-letter count (target: zero).

PART II

API Reference

8 · CONVENTIONS

Read this first

Envelope. Every response, success or failure, is `{ "isSuccess": bool, "data": ... | null, "message": string }` — the message is always populated and safe to show in the UI. HTTP status codes carry the semantics.

Auth. Protected endpoints take `Authorization: Bearer <accessToken>`. Tokens come from `verify-auth` (during onboarding) or `login`, and refresh via `/api/auth/refresh`. The badge on each endpoint shows who may call it.

OTP ceremonies. Every `initiate-*` returns a `retrievalCode`; quote it back together with the 6-digit `otp` the user received. Codes expire in 10 minutes, lock after 5 wrong attempts, and resend is throttled to once per 60 seconds.

Payload encryption (when `Encryption:Enabled`): request and response bodies travel as `{ "data": "base64(noncelciphertextlltag)" }`, AES-256-GCM with the shared key. `/health`, `/swagger`, `/openapi` stay plaintext.

ANON no token ANY any valid token ONBOARDING onboarding type only

PROFILE-ACTIVE profile-active type only

GET /health

ANON

Liveness probe; also proves the envelope shape. Reports the credit-posting job's heartbeat — a stalled job flips `status` to `degraded`.

RESPONSE 200

```
{ "isSuccess": true,
  "data": {
    "status": "healthy", // or "degraded"
    "creditJob": {
      "name": "cba-credit-post",
      "enabled": true,
      "lastTickUtc": "...",
      "ticks": 120491,
      "healthy": true } },
  "message": "Operation successful." }
```

9 · ONBOARDING

Onboarding

The sequence: initiate → verify-auth → initiate-phone (or initiate-kyc → otp/verify · Phone → complete-kyc) → create-profile. Recovery: initiate-resume → resume. The KYC pair also works *after* the profile is active — the app's phone-first journey.

POST /api/onboarding/initiate

ANON

Start onboarding: checks the email is new (locally and on the SSO), registers/refreshes the device, pre-allocates the identity, and emails the OTP. Send full `device` details the first time; a known `deviceId` (or both) afterwards. Calling again resends a fresh code.

REQUEST

```
{ "emailAddress": "ada@example.com",
  "deviceId": null,
  "device": {
    "deviceIdentifier": "install-abc-123",
    "name": "Ada's iPhone",
    "platform": "ios",
    "osName": "iOS", "osVersion": "18.2",
```

```
"manufacturer": "Apple", "model": "iPhone 16",
"appVersion": "1.0.0" } }
```

RESPONSE 202

```
{ "emailAddress": "ada@example.com",
  "deviceId": "GUID",
  "retrievalCode": "GKSLW7AVA8YJAWAM",
  "expiresAtUtc": "..." }
// message: "OTP sent to your email address."
```

409 profile or SSO account already exists (points at resume) · **422** unknown deviceId with no device info · **429** resend cooldown · **503** SSO unreachable

POST /api/onboarding/verify-auth

ANON

Verify the email OTP, create the **SSO auth** (SourceId = profile id) and the staging record (AuthCreated , email confirmed, device bound) — then log the new account in and return onboarding-typed tokens, so the flow continues without a login call.

REQUEST

```
{ "emailAddress": "ada@example.com",
  "deviceId": "GUID",
  "retrievalCode": "GKSL...",
  "otp": "123456",
  "password": "S3cure-Pass!",
  "username": null }
```

RESPONSE 201

```
{ "profileId": "GUID",
  "emailAddress": "ada@example.com",
  "emailConfirmed": true,
  "status": "AuthCreated",
  "type": "onboarding",
  "deviceId": "GUID",
  "createdAtUtc": "...",
  "tokens": {
    "accessToken": "eyJ...", "refreshToken": "...",
    "idToken": "...", "tokenType": "Bearer",
    "expiresIn": 3599 } }
```

400 wrong / expired / wrong-purpose / wrong-device code · **409** profile already exists · **429** attempt
lockout · **502/503** SSO rejection / unreachable

Password rules: min 8, uppercase, lowercase, number, special character — each failure gets its own message.

POST /api/onboarding/initiate-phone

 **ONBOARDING**

Phone leg: the profile is resolved from the device's active binding — no profile id needed. Stores the phone (unconfirmed, uniqueness-checked) and sends the OTP over WhatsApp.

REQUEST

```
{ "phoneNumber": "+2348060168634",  
  "deviceId": "GUID" }
```

RESPONSE 202

```
{ "profileId": "GUID",  
  "phoneNumber": "+2348060168634",  
  "channel": "whatsapp",  
  "retrievalCode": "...", "expiresAtUtc": "..." }
```

403 wrong token type · **409** phone already confirmed / used by another profile · **422** device not linked to any profile · **429** cooldown · **502** send failed

POST /api/onboarding/initiate-kyc

 **ANY**

KYC start, valid before *or* after the profile is active: submit **either** `bvn` **or** `nin` (11 digits). The identity record (names, DOB, gender, photo) is looked up, the profile pre-filled from it, and an **AWS face-liveness session** opened for the client SDK. When the phone is not yet confirmed, it is adopted from the record — never from the client — and an OTP goes to it (`otp` is `null` when the phone is already confirmed). The response reveals only a masked number.

REQUEST

```
{ "deviceId": "GUID",  
  "bvn": "22233344455",  
  "nin": null }
```

RESPONSE 200

```
{ "profileId": "GUID",  
  "idType": "bvn",  
  "kycStatus": "Pending",  
  "identity": {  
    "firstName": "Ada", "lastName": "Obi",  
    "middleName": null,
```

```

    "dateOfBirth": "1995-05-15", "gender": "Female",
    "maskedPhoneNumber": "*****8634",
    "image": "base64..." },
    "liveness": { "sessionId": "...", "authToken": "..." },
    "otp": {                                // null when phone already confirmed
      "channel": "WhatsApp",
      "retrievalCode": "...", "expiresAtUtc": "..." } }

```

400 both/neither id supplied · **403** device not linked · **409** id already verified / id or phone used by another profile · **422** no record for this BVN/NIN, or email not yet confirmed · **429** cooldown · **502/503** KYC or liveness provider errors

POST /api/onboarding/complete-kyc

 **ANY**

Finish KYC after the client SDK runs the liveness session: fetches the liveness result, compares the live selfie with the BVN/NIN photo, and on a match marks the id verified, sets `kycStatus: Approved`, promotes to **tier 1** — and **provisions the banking accounts** (CIF → NGN → CAD → virtual). A failed liveness or face mismatch is a *soft* failure: HTTP 200 with `isLive` / `faceMatch` false — retry with a fresh session from `initiate-kyc`.

REQUEST

```

{ "deviceId": "GUID",
  "sessionId": "..." } // from initiate-kyc

```

RESPONSE 200

```

{ "profileId": "GUID",
  "idType": "bvn",
  "livenessScore": 0.98, "isLive": true,
  "faceMatch": true, "faceMatchConfidence": 99.2,
  "bvnIsVerified": true, "ninIsVerified": false,
  "phoneNumberConfirmed": true,
  "tier": 1,
  "kycStatus": "Approved", // or ProvisioningFailed
  "cif": "0012345",
  "nairaAccount": "1000012345",
  "cadAccount": "1000012346",
  "virtualAccount": "8812345678",
  "virtualAccountBank": "Digitvant Microfinance Bank Ltd" }

```

403 device not linked / no profile · **409** identity already verified · **422** no BVN/NIN staged (run `initiate-kyc` first) · **502/503** liveness, KYC or comparison provider errors

Account provisioning is best-effort: on a provider outage the response shows `kycStatus: ProvisioningFailed` and the missing accounts are created automatically at `create-profile`, `login` or `refresh`.

POST /api/onboarding/create-profile

 **ONBOARDING**

The final step — creates the profile proper. Identity comes from the token (SourceId claim, email fallback); the `deviceId` must match the active binding; email *and* phone must both be confirmed. Flips status to `Active` and the SSO type claim to `profile-active`, and retries any banking-account provisioning still missing from KYC.

REQUEST

```
{ "deviceId": "GUID",
  "firstName": "Ada",
  "lastName": "Obi",
  "middleName": null,
  "dateOfBirth": "1995-05-15" }
```

RESPONSE 200

```
{ "profileId": "GUID",
  "firstName": "Ada", "lastName": "Obi",
  "middleName": null,
  "dateOfBirth": "1995-05-15",
  "status": "Active",
  "updatedAtUtc": "..." }
```

400 invalid date of birth · **403** device not linked / token resolves to no profile / wrong token type ·
422 email or phone not confirmed yet

POST /api/onboarding/initiate-resume

 **ANY**

Recovery bootstrap for an SSO account with no local record (interrupted verify-auth, or a user from another system). Log in first — the password proves the account — then this sends a **device-approval OTP** to the account email. Empty body.

REQUEST

```
{ }
```

RESPONSE 202

```
{ "emailAddress": "ada@example.com",
  "channel": "email",
  "retrievalCode": "...", "expiresAtUtc": "..." }
```

400 token has no email claim · **409** a profile already exists — nothing to resume · **429** cooldown

POST /api/onboarding/resume

 **ANY**

Verifies the resume OTP, registers the device (this OTP is its approval — the only way a resume device is registered), and recreates the staging record **under the token's original SourceId**. Onboarding then continues at the phone leg. Idempotent when a record already exists.

REQUEST

```
{ "retrievalCode": "...",  
  "otp": "123456",  
  "deviceId": null,  
  "device": { ...full device details... } }
```

RESPONSE 201

```
{ "profileId": "GUID",    // = SourceId  
  "emailAddress": "ada@example.com",  
  "status": "AuthCreated",  
  "deviceId": "GUID",  
  "resumed": true }
```

400 bad/expired code, wrong purpose or account · **409** device actively in use by another profile (victim untouched) · **422** unknown deviceId with no details · **429** logout

10 · OTP VERIFICATION

The single verify endpoint

POST /api/otp/verify

 **ANY**

One endpoint verifies every plain confirmation; the `section` enum selects what gets confirmed. `Email` confirms the email address; `Phone` confirms the phone — including the phone adopted from a BVN/NIN record during KYC (the identity itself is verified biometrically at `complete-kyc`, not by an OTP). Already-verified sections return success idempotently. (Flows with extra inputs — `verify-auth`, `pin-reset`, `password-reset`, `change-device`, `resume` — keep their own endpoints.)

REQUEST

```
{ "deviceId": "GUID",  
  "retrievalCode": "...",  
  "otp": "123456",  
  "section": "Phone"    // Email | Phone  
}
```

RESPONSE 200

```
{ "profileId": "GUID",
  "section": "Phone",
  "emailConfirmed": true,
  "phoneNumberConfirmed": true,
  "bvnIsVerified": false,
  "ninIsVerified": false,
  "tier": 0,
  "updatedAtUtc": "..." }
// message per section, e.g. "Phone number verified."
```

400 unknown section, wrong code, wrong purpose/profile, contact changed since issue · **403** device not linked · **422** no phone on the profile · **429** logout

11 · AUTH

Sessions, passwords, device change

POST /api/auth/login

ANON

SSO password grant gated on the device: the `deviceId` must be registered, and if the profile is bound to a *different* device the login is refused — the device-change ceremony is the only way to move. Returns tokens, a `deviceStatus`, the token `type`, the full profile DTO (null for SSO-only accounts mid-resume) — and, for a provisioned profile, the **live account balances and latest exchange rates**. Any missing accounts are re-provisioned first (the self-heal); provider outages yield empty lists, never a failed login.

REQUEST

```
{ "username": "ada@example.com",
  "password": "S3cure-Pass!",
  "deviceId": "GUID" }
```

RESPONSE 200

```
{ "accessToken": "eyJ...",
  "refreshToken": "...", "idToken": "...",
  "tokenType": "Bearer", "expiresIn": 3599,
  "deviceStatus": "Existing", // New | Existing | Unlinked
  "type": "profile-active",
  "profile": { ...profile DTO, see /auth/me... },
  "accounts": [
    { "accountName": "ADA OBI", "accountNumber": "1000012345",
      "currency": "NGN", "balance": 25000.50 },
    { "accountName": "ADA OBI", "accountNumber": "1000012346",
      "currency": "CAD", "balance": 120.00 } ],
  "rates": [
```

```
{ "sourceCode": "CAD", "targetCode": "NGN", "type": "Transfer",  
  "buyRate": 1080.0, "sellRate": 1100.0,  
  "date": "...", "updatedAtUtc": "..." } ] }
```

401 wrong credentials · 403 profile bound to a different device · 422 unregistered deviceId · 503 SSO unreachable

POST /api/auth/refresh

ANON

Exchanges the refresh token and, like login, resolves the profile from the new token, retries any missing account provisioning, and returns the profile DTO with accounts and rates.

REQUEST

```
{ "token": "<refreshToken>" }
```

RESPONSE 200

```
{ "accessToken": "...", "refreshToken": "...",  
  "idToken": "...", "tokenType": "Bearer",  
  "expiresIn": 3599,  
  "type": "profile-active",  
  "profile": { ...profile DTO... },  
  "accounts": [ ... ], "rates": [ ... ] }
```

401 refresh token invalid or expired

GET /api/auth/me

 ANY

Reload the caller's profile from the token alone — the app-relaunch pattern. Same DTO as login.

RESPONSE 200 – THE PROFILE DTO

```
{ "profileId": "GUID", "status": "Active",  
  "emailAddress": "ada@example.com", "phoneNumber": "+2348060168634",  
  "firstName": "Ada", "lastName": "Obi", "middleName": null,  
  "gender": "Female", "dateOfBirth": "1995-05-15",  
  "tier": 1, "address": "12 Marina Road, Lagos",  
  "cif": "0012345",  
  "nairaAccount": "1000012345", "cadAccount": "1000012346",  
  "virtualAccount": "8812345678",  
  "virtualAccountBank": "Digitvant Microfinance Bank Ltd",  
  "kycStatus": "Approved", "kycStatusReason": null,  
  "bvn": "*****4455", "nin": null,  
  "emailConfirmed": true, "phoneNumberConfirmed": true,  
  "bvnIsVerified": true, "ninIsVerified": false,  
  "hasSetTransactionPin": true, "profileCompleted": true,
```

```
"activeDeviceId": "GUID",
"activeDeviceLinkedAtUtc": "...", "deviceRecentlyLinked": false,
"deviceChangedAtUtc": null, "deviceRecentlyChanged": false,
"createdAtUtc": "..." }
```

401 no/invalid token · **403** token resolves to no profile

`deviceRecentlyLinked` is the **transaction-limit gate**: true while the active device binding is under 24 hours old.

POST /api/auth/password-change

 **PROFILE-ACTIVE**

REQUEST

```
{ "deviceId": "GUID",
  "currentPassword": "...",
  "newPassword": "..." }
```

RESPONSE 200

```
{ "profileId": "GUID",
  "message": "Password changed successfully." }
```

400 current password wrong / new password fails complexity · **403** device or type gate · **503** SSO unreachable

POST /api/auth/initiate-password-reset

 **PROFILE-ACTIVE**

REQUEST

```
{ "deviceId": "GUID" }
```

RESPONSE 202

```
{ "profileId": "GUID", "channel": "email",
  "retrievalCode": "...", "expiresAtUtc": "..." }
```

403 gates · **429** cooldown · **502** send failed

POST /api/auth/password-reset

 **PROFILE-ACTIVE**

Verifies the OTP, then obtains the SSO's reset token server-to-server and sets the new password.

REQUEST

```
{ "deviceId": "GUID",  
  "retrievalCode": "...", "otp": "123456",  
  "newPassword": "..." }
```

RESPONSE 200

```
{ "profileId": "GUID",  
  "message": "Password reset successfully." }
```

400 bad code / weak password · **429** logout · **502/503** SSO errors

POST /api/auth/initiate-device-change

ANON

Built for the **locked-out user** — a new phone can't log in (403) and holds no token or profile id. The password is the first factor, verified against the SSO before any OTP is sent; the profile resolves from the credentials.

REQUEST

```
{ "username": "ada@example.com",  
  "password": "S3cure-Pass!" }
```

RESPONSE 202

```
{ "channel": "email",  
  "retrievalCode": "...", "expiresAtUtc": "..." }
```

401 wrong password (no OTP sent) · **422** no profile for this account — use resume · **429** cooldown

POST /api/auth/change-device

ANON

The mailbox OTP is the second factor. Registers the replacement device (details for a brand-new one, or a known `deviceId`) and moves the exclusive binding in one step — the old device's row is released to history and its logins start failing. The fresh `linked_at` opens the 24-hour reduced-limit gate.

REQUEST

```
{ "username": "ada@example.com",  
  "retrievalCode": "...", "otp": "123456",  
  "deviceId": null,  
  "device": { ...full device details... } }
```

RESPONSE 200

```
{ "deviceId": "GUID",
  "deviceChangedAtUtc": "...",
  "deviceRecentlyLinked": true }
// "Device changed successfully. You can now
// log in from this device."
```

400 bad code / wrong account · **422** no profile, or unknown deviceId with no details · **429** logout

12 · PROFILE

Reads, PIN, audit

GET /api/profiles/{id}

ANON

Full profile by id — the same fields as the [/auth/me](#) DTO plus `deviceId` (the creating device) and `updatedAtUtc`. BVN/NIN always masked. 404 when not found.

GET /api/profiles?page=&pageSize=

ANON

Paged listing (pageSize clamped to 100): `{ items: [{id, emailAddress, phoneNumber, emailConfirmed, phoneNumberConfirmed, deviceId, createdAtUtc}], page, pageSize, totalCount }`.

GET /api/profiles/devices

 ANY

The caller's own devices over time — profile resolved from the token, never from the URL. Exactly one entry has `isActive: true`.

RESPONSE 200

```
[ { "deviceId": "GUID", "name": "Ada's iPhone", "platform": "ios",
  "model": "iPhone 16", "isActive": true,
  "linkedAtUtc": "...", "releasedAtUtc": null },
  { "deviceId": "GUID", "name": "Old Pixel", "platform": "android",
  "model": "Pixel 9", "isActive": false,
  "linkedAtUtc": "...", "releasedAtUtc": "..."} ]
```

GET /api/profiles/activities?page=&pageSize=

 **ANY**

The caller's audit feed, newest first. Types: AuthCreated, ProfileCreated, EmailVerified, PhoneNumberSet, PhoneVerified, ProfileCompleted, LoggedIn, PasswordChanged, PasswordReset, PinSet, PinChanged, PinReset, DeviceChanged, KycVerified, AccountsProvisioned.

RESPONSE 200

```
{ "items": [ { "id": "GUID", "profileId": "GUID",
  "type": "DeviceChanged",
  "description": "Active device changed to 'Ada's iPhone'.",
  "deviceId": "GUID", "ipAddress": "105.112.24.8",
  "createdAtUtc": "..." } ],
  "page": 1, "pageSize": 20, "totalCount": 42 }
```

POST /api/profiles/set-pin

 **PROFILE-ACTIVE**

First-time 4-digit transaction PIN. Stored as PBKDF2-SHA256 with a fresh per-profile salt; verified during transactions, not by this API. Once set, only pin-change or pin-reset may alter it.

REQUEST

```
{ "deviceId": "GUID", "pin": "1234" }
```

RESPONSE 200

```
{ "profileId": "GUID",
  "hasSetTransactionPin": true,
  "updatedAtUtc": "..." }
```

400 not 4 digits · **403** device or type gate · **409** PIN already set

POST /api/profiles/pin-change

 **PROFILE-ACTIVE**

REQUEST

```
{ "deviceId": "GUID",
  "currentPin": "1234", "newPin": "5678" }
```

RESPONSE 200

```
{ "profileId": "GUID",
  "hasSetTransactionPin": true,
```

```
"updatedAtUtc": "..." }
```

400 current PIN incorrect · **422** no PIN set yet

POST /api/profiles/initiate-pin-reset

 **PROFILE-ACTIVE**

Forgot-PIN path: emails an OTP. Refuses if no PIN is set.

REQUEST

```
{ "deviceId": "GUID" }
```

RESPONSE 202

```
{ "profileId": "GUID", "channel": "email",  
  "retrievalCode": "...", "expiresAtUtc": "..." }
```

POST /api/profiles/pin-reset

 **PROFILE-ACTIVE**

REQUEST

```
{ "deviceId": "GUID",  
  "retrievalCode": "...", "otp": "123456",  
  "newPin": "4321" }
```

RESPONSE 200

```
{ "profileId": "GUID",  
  "hasSetTransactionPin": true,  
  "updatedAtUtc": "..." }
```

400 bad code · **429** logout

PUT /api/profiles/{id}/phone

ANON

Post-onboarding phone *change*: sets a new number (uniqueness-checked, resets confirmation). Follow with `request-phone-otp` + `otp/verify` · Phone . Body: `{ "phoneNumber", "deviceId" }` → `{ id, phoneNumber, phoneNumberConfirmed:false, updatedAtUtc }` . 409 if another profile uses the number.

POST /api/profiles/{id}/request-email-otp · .../request-phone-otp

ANON

Re-confirmation OTPs for an existing profile — email by email, phone by WhatsApp. Empty body. Both return { `profileId`, `channel`, `retrievalCode`, `expiresAtUtc` }; verify via `otp/verify`. 409 when already confirmed; 422 when no phone is set.

GET /api/devices/{id}

ANON

Device lookup: { `id`, `deviceIdentifier`, `name`, `platform`, `osName`, `osVersion`, `manufacturer`, `model`, `appVersion`, `createdAtUtc`, `updatedAtUtc` }. Registration only ever happens inside the OTP-protected flows.

13 · WEBHOOKS

Money in

POST /webhook/virtual-account-credit

ANON + HMAC

Called by the virtual-account provider when a customer's collection account is credited. When `DigitVirtual:WebhookSecret` is configured, the `X-Signature` header must be the lowercase-hex **HMAC-SHA256 of the raw body** (fixed-time compared); when unset (dev only), verification is skipped. The credit is stored idempotently by `transactionRef` — the `PostCbaCreditsJob` then settles it into the mapped CBA naira account, minus the configured charges. A storage failure returns 500 so the provider redelivers rather than losing the credit; a malformed payload is acknowledged and dropped (redelivery cannot fix it).

REQUEST

```
{ "transactionRef": "VA-2026-000123",
  "amount": 15000.00,
  "account": "8812345678",
  "sourceAccount": "0123456789",
  "sourceBank": "GTBank",
  "senderName": "CHIDI OKafor",
  "narration": "rent",
  "transactionDate": "...",
  "status": "SUCCESSFUL" }
```

RESPONSE 200

```
{ "status": "received",
  "receivedAtUtc": "..." }
// or { "status": "ignored", ... }
//     for malformed payloads
```

401 invalid signature · **500** storage failure — provider should redeliver

Only credits with `status: SUCCESSFUL` are settled. Transient posting failures retry every ~2 s up to `MaxPostAttempts` (default 10); permanent 4xx rejections dead-letter with the error preserved; a CBA "duplicate reference" reply counts as settled.

14 · APPENDIX

Reference tables

OTP purposes

PURPOSE	SENT VIA	TARGET	VERIFIED BY	ON SUCCESS
Onboarding	email	email	verify-auth	SSO auth + staging record + tokens
PhoneConfirmation	WhatsApp/SMS	phone	otp/verify · Phone	phone confirmed
EmailConfirmation	email	email	otp/verify · Email	email confirmed
PhoneConfirmation (KYC path)	WhatsApp/SMS	phone from BVN/NIN record	otp/verify · Phone	phone confirmed (id verified separately at complete-kyc)
DeviceChange	email	email	change-device	binding moved · gate opens
DeviceRegistration	email	email	resume	device registered · record rebuilt
PinReset	email	email	pin-reset	new PIN hash
PasswordReset	email	email	password-reset	SSO password reset

KYC statuses

KYCSTATUS	MEANING	RECOVERABLE?
NotStarted	No KYC attempt yet	—

KYCSTATUS	MEANING	RECOVERABLE?
Pending	Identity looked up, liveness session open, awaiting complete-kyc	in progress
Approved	Liveness + face match passed; identity verified, accounts provisioned	terminal (good)
LivenessFailed	The liveness check did not pass	yes — new session
FaceMismatch	The selfie did not match the BVN/NIN photo	yes — new session
ProvisioningFailed	Identity verified but account setup incomplete	auto-retried at create-profile / login / refresh
Retry / Rejected / Failed	Provider asked for resubmission / permanently rejected / could not complete	Retry: yes · Rejected: no · Failed: yes

Status codes

CODE	MEANING HERE
200 / 201 / 202	Success · created · accepted (OTP dispatched)
400	Validation failure, wrong OTP, malformed body — the message names the field or rule
401	Missing/invalid token, or wrong credentials at login
403	Wrong token type for this endpoint, device not the active one, or token resolves to no profile
404	Profile or device not found
409	Conflict: already exists / already confirmed / device in use / PIN already set
422	Preconditions unmet: unknown device, unconfirmed phone, no KYC record, nothing to resume
429	Resend cooldown (60 s) or OTP attempt lockout (5 tries)
502 / 503	A dependency (SSO, Message Centre, KYC, liveness, core banking) rejected the call or is unreachable — nothing was persisted

